

EXCEL ENGINEERING COLLEGE

(Autonomous)

Approved by AICTE, New Delhi & Affiliated to Anna University,
Chennai Accredited by NBA, NAAC with “A+” and Recognized
byUGC (2f &12B) KOMARAPALAYAM - 637303



DEPARTMENT OF INFORMATION TECHNOLOGY

**23AI502 - MACHINE LEARNING
LABORATORY**

VI SEMESTER



**ENGINEERING COLLEGE
(AUTONOMOUS)
KOMARAPALAYAM-637 303**



**DEPARTMENT OF INFORMATION TECHNOLOGY
LABORATORY RECORD NOTE BOOK**

This is to certify that bonafide record of work done by _____

Register No. _____ 6th semester of Computer Science and
Engineering

Branch during the year 2025-2026 in the _____
LABORATORY.

Staff In-charge

Head of the Department

Submitted for the Practical Examination held on: _____

Internal Examiner

External Examiner

INDEX

S. NO	DATE	EXPERIMENTS	PAGE NO	MARKS	SIGN
1		FIND S ALGORITHM			
2		CANDIDATE ELIMINATION ALGORITHM			
3		ID3			
4		BACKPROPAGATION ALGORITHM			
5		NAÏVE BAYES CLASSIFIER			
6		DOCUMENT CLASSIFICATION USING NAÏVE BAYES CLASSIFIER			
7		BAYESIAN NETWORK			
8		CLUSTERING BASED ON EXPECTATIONS MAXIMUM AND K-MEANS ALGORITHM			
9		KNN ALGORITHM			
10		LOCALLY WEIGHTED REGRESSION ALGORITHM			

Internal Marks: -----/25

Staff In-charge

EXCEL ENGINEERING COLLEGE

VISION

- To create competitive human resources in the fields of engineering for the benefit of society to meet global challenges.

MISSION

- To provide a conducive ambience for better learning and to bring creativity in the students.
- To develop sustainable environment for innovative learning to serve the needy.
- To meet global demands for excellence in technical education.
- To train young minds with values, culture, integrity, innovation and leadership.

DEPARTMENT OF INFORMATIONTECHNOLOGY

VISION

- To craft professional technocrats with ethically strong human values to cater the global needs in the field of Information Technology.

MISSION

- To provide technical education to the students for achieving professional career and high education
- To promote entrepreneurship initiatives among stakeholders.
- To initiate research cultures and innovation.
- To create ambience for achieving societal needs.

DEPARTMENT OF INFORMATION TECHNOLOGY

Program Educational Objectives (PEOs)

PEO - I

To ensure graduates for utilizing the fundamental knowledge of sciences, mathematics and Information Technology for the applications relevant to various streams of Engineering and Technology.

PEO-II

To enrich graduates with the core competencies necessary for applying knowledge of computers and telecommunications equipment to store, retrieve, transmit, manipulate and analyze data in the context of business enterprise.

PEO-III

To enable graduates to think logically, pursue lifelong learning and will have the capacity to understand technical issues related to computing systems and to design optimal solutions.

PEO-IV

To enable graduates to develop hardware and software systems by understanding the importance of social, business and environmental needs in the human context.

PEO-V

To enable graduates to gain employment in organizations and establish themselves as professionals by applying their technical skills to solve real world problems and meet the diversified needs of industry, academia and research.

PROGRAMME OUTCOMES (POs)

1. **Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
2. **Problem Analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design / Development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods, including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling of complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and Sustainability:** Understand the impact of the professional engineering solutions to societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and

writes effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of the engineering management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Lifelong learning: Recognize the need for and have the preparation and ability to engage in independent and lifelong.

Mapping of Course Outcomes (COs) with Programme Outcomes (POs) Programme Specific Outcomes(PSOs)															
COs	POs												PSOs		
	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
CO1	3	2	1	1									3	1	
CO2	3	2	2	1	2						2	2	3	1	
CO3	3	2	1	1									3	1	
CO4	3	3	2	1	2	1		1			1	2	3	1	
CO5	3	3	2	2	2	1		1			3	2	3	1	
	3	High				2	Medium					1	Low		

Exp.No:

Date:

FIND-S ALGORITHM

LEARNING OBJECTIVES:

- Make use of Data sets in implementing the machine learning algorithms.
- Implement ML concepts and algorithms in Python

AIM:

To Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a .CSVfile

THEORY:

- The concept learning approach in machine learning can be formulated as –Problem of searching through a predefined space of potential hypotheses for the hypothesis that best fits the training examples.
- Find-S algorithm for concept learning is one of the most basic algorithms of machine learning.

Find-S Algorithm:

- Initialize h to the most specific hypothesis in H
- For each positive training instance x
- For each attribute constraint a i in h :
- If the constraint a i in h is satisfied by x then do nothing
- Else replace a i in h by the next more general constraint that is satisfied by x
- Output hypothesis h
- It is guaranteed to output the most specific hypothesis within H that is consistent with the positivetraining examples.
- Also Notice that negative examples are ignored.

Dataset (Data.csv)

Example,Sky,Temp,Humidity,Wind,Water,Forecast,Enjoysport
1,Sunny,Warm,Normal,Strong,Warm,Same,Yes
2,Sunny,Warm,High,Strong,Warm,Same,Yes
3,Rainy,Cold,High,Strong,Warm,Change,No
4,Sunny,Warm,High,Strong,Cool,Change,Yes

PROGRAM:

```
In [1]: import pandas as pd
import numpy as np
```

```
In [2]: d = pd.read_csv("Data.csv")
```

```
In [3]: print(d)
```

	Example	Sky	Temp	Humidity	Wind	Water	Forecast	Enjoysport
0	1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	2	Sunny	Warm	High	Strong	Warm	Same	Yes
2	3	Rainy	Cold	High	Strong	Warm	Change	No
3	4	Sunny	Warm	High	Strong	Cool	Change	Yes

```
In [11]: a = np.array(d)[:,-1]
print(" The attributes are: ",a)
```

```
The attributes are: [[1 'Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
[2 'Sunny' 'Warm' 'High' 'Strong' 'Warm' 'Same']
[3 'Rainy' 'Cold' 'High' 'Strong' 'Warm' 'Change']
[4 'Sunny' 'Warm' 'High' 'Strong' 'Cool' 'Change']]
```

```
In [12]: t = np.array(d)[:, -1]
print("The target is: ",t)
```

```
The target is: ['Yes ' 'Yes ' 'No ' 'Yes']
```

```
In [25]: def train(c,t):
    for i, val in enumerate(t):
        if val == "Yes":
            break
    specific_hypothesis = c[i].copy()
    for i, val in enumerate(c):
        if t[i] == "Yes":
            for x in range(len(specific_hypothesis)):
                if val[x] != specific_hypothesis [x]:
                    specific_hypothesis [x] = '?'
            else:
                pass
    return specific_hypothesis
```

```
In [26]: print(" The final hypothesis is: ",train(a,t))
```

```
The final hypothesis is: ['?' '?' '?' 'High' 'Strong' '?' 'Change']
```

RESULT:

Thus the Implementation for Find S Algorithm using Python Program has been executed successfully and output also verified

Exp.No:

Date:

CANDIDATE ELIMINATION ALGORITHM

LEARNING OBJECTIVES:

- Make use of Data sets in implementing the machine learning algorithms.
- Implement ML concepts and algorithms in Python.

AIM:

To implement and demonstrate the Candidate-Elimination algorithm to output a Description of the set of all hypotheses consistent with the training examples.

THEORY:

- The key idea in the Candidate-Elimination algorithm is to output a description of the set of all hypotheses consistent with the training examples.
- It computes the description of this set without explicitly enumerating all of its members.
- This is accomplished by using the more-general-than partial ordering and maintaining a compact representation of the set of consistent hypotheses.
- The algorithm represents the set of all hypotheses consistent with the observed training examples. This subset of all hypotheses is called the version space with respect to the hypothesis space H and the training examples D , because it contains all plausible versions of the target concept.
- A version space can be represented with its general and specific boundary sets.
- The Candidate-Elimination algorithm represents the version space by storing only its most general members G and its most specific members S .
- Given only these two sets S and G , it is possible to enumerate all members of a version space by generating hypotheses that lie between these two sets in general-to-specific partial ordering over hypotheses. Every member of the version space lies between these boundaries

ALGORITHM:

1. Initialize G to the set of maximally general hypotheses in H
2. Initialize S to the set of maximally specific hypotheses in H
3. For each training example d , do If d is a positive example
4. Remove from G any hypothesis inconsistent with d , For each hypothesis s in S that is not consistent with d ,

5. Remove s from S
6. Add to S all minimal generalizations h of s such that h is consistent with d , and some member of G is more general than h
7. Remove from S , hypothesis that is more general than another hypothesis in S If d is a negative example
8. Remove from S any hypothesis inconsistent with d For each hypothesis g in G that is not consistent with d
9. Remove g from G
10. Add to G all minimal specializations h of g such that h is consistent with d , and some member of S is more specific than h .
11. Remove from G any hypothesis that is less general than another hypothesis in G

DATASET: (Data.csv)

Example, Sky, Temp, Humidity, Wind, Water, Forecast, Enjoyment

1, Sunny, Warm, Normal, Strong, Warm, Same, Yes

2, Sunny, Warm, High, Strong, Warm, Same, Yes

3, Rainy, Cold, High, Strong, Warm, Change, No

4, Sunny, Warm, High, Strong, Cool, Change, Yes

PROGRAM:

```
import
numpy as
npimport
pandas as
pd
data =
pd.DataFrame(data=pd.read_csv('Data.csv'
))concepts = np.array(data.iloc[:,0:-1])
print(concepts)
target =
np.array(data.iloc[:,-1])
print(target)

def learn(concepts,
target): specific_h =
concepts[0].copy()
print("Initialization of specific_h and
general_h")print(specific_h)
general_h = [ "?" for i in range(len(specific_h))] for i in
range(len(specific_h))print(general_h)

for i, h in
enumerate(concepts)
:if target[i] ==
```

```

    "Yes":
        for x in
            range(len(specific_
h)):if h[x] !=
specific_h[x]:
            specific_
h[x] = '?'
            general_h[x
][x] = '?'
        print(specific_h)

    if target[i] == "No":
        for x in
            range(len(specific_
h)):if h[x] !=
specific_h[x]:
            general_h[x][x] =
specific_h[x]else:
            general_h[x][x] = '?'
        print("Steps of Candidate Elimination
Algorithm", i + 1)print(specific_h)
        print(general_h)

indices = [i for i, val in enumerate(general_h) if val == ['?' for _ in
range(len(specific_h))]]for i in indices:
    general_h.remove(['?' for _ in range(len(specific_h))])

    return specific_h, general_h
s_final, g_final =
learn(concepts, target)
print("Final Specific_h:")
print(s_final)

print("Final
General_h:")
print(g_final)

```

OUTPUT:

```

Initialization of specific_h and general_h
['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
[['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?',
 '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]
['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
['Sunny', 'Warm', '?', 'Strong', 'Warm',
'Same']Steps of Candidate
Elimination Algorithm 3 ['Sunny',
'Warm', '?', 'Strong', 'Warm', 'Same']
[['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?',

```

```
'Strong', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', 'Same']]
['Sunny', 'Warm', '?',
'Strong', '?', '?']Final
Specific_h:
['Sunny', 'Warm', '?',
'Strong', '?', '?']Final
General_h:
[]
```

RESULT:

Thus the Implementation for Candidate Algorithm using Python Program has been executed successfully and output also verified

Exp.No:

Date:

ID3

LEARNING OBJECTIVES:

- Make use of Data sets in implementing the machine learning algorithms. Implement ML concepts and algorithms in Python.

AIM:

To write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

THEORY:

- ID3 algorithm is a basic algorithm that learns decision trees by constructing them top down, beginning with the question "which attribute should be tested at the root of the tree?".
- To answer this question, each instance attribute is evaluated using a statistical test to determine how well it alone classifies the training examples. The best attribute is selected and used as the test at the root node of the tree.
- A descendant of the root node is then created for each possible value of this attribute, and the training examples are sorted to the appropriate descendant node (i.e., down the branch corresponding to the example's value for this attribute).
- The entire process is then repeated using the training examples associated with each descendant node to select the best attribute to test at that point in the tree.
- A simplified version of the algorithm, specialized to learning Boolean-valued functions (i.e., concept learning), is described below.

Algorithm:

ID3(Examples, Target Attribute, Attributes)Input:

Examples are the training examples.

Target attribute is the attribute whose value is to be predicted by the tree.

Attributes is a list of other attributes that may be tested by the learned decision tree.

Output: Returns a decision tree that correctly classifies the given Examples Method:

1. Create a Root node for the tree
2. If all Examples are positive, Return the single-node tree Root, with label = +

3. If all Examples are negative, Return the single-node tree Root, with label = -

4. If Attributes is empty,

Return the single-node tree Root, with label = most common value of Target

Attribute in Examples

Else

A ← the attribute from Attributes that best classifies

Examples The decision attribute for Root ← A

For each possible value, v_i , of A,

Add a new tree branch below Root, corresponding to the test $A = v_i$ Let Examples_{v_i} be the subset of Examples that have value v_i for A

If Examples_{v_i} is empty Then below this new branch add a leaf node with label = most common value of Target Attribute in Examples

Else below this new branch add the sub tree ID3 (Examples_{v_i} , Target Attribute, Attributes – {A})

End Return Root

PROGRAM:

```
import
numpy as
npimport
math
import csv

def read_data(tennisdata):
    with open(tennisdata, 'r') as csvfile:
        datareader = csv.reader(csvfile,
            delimiter=',')
        headers = next(datareader)
        meta
        data
        = []
        traind
        ata =
        []
        for name in
            headers:
                metadata.append
                    d(name)
        for row in
            datareader:
```

```

        traindata.append(
            row)
    return (metadata,
traindata)
class Node:
    def __init__(self,
        attribute):
        self.attribute =
            attribute
        self.children = []
        self.answer = ""

    def __str__(
        self):
        return
            self.attribute

def subtables(data, col,
    delete):
    items =
        np.unique(data[:, col])
    count = np.zeros((items.shape[0], 1),
        dtype=np.int32)
    for x in
        range(items.shape[0]):
            for y in
                range(data.shape[0]
                    ):
                if data[y, col] ==
                    items[x]:
                        count[
                            x] += 1
    dictionary =
        {}
    for x in range(items.shape[0]):
        dictionary[items[x]] = np.empty((int(count[x]), data.shape[1]),
            dtype="<S32")
        pos = 0
        for y in
            range(data.shape[0]
                ):
                if data[y, col] ==
                    items[x]:
                        dictionary[items[x]][pos] =
                            data[y]
                        pos += 1
        if delete:
            dictionary[items[x]] =
                np.delete(dictionary[items[x]], col, 1)
    return items,
        dictionary

def entropy(S):

```

```

items =
np.unique(S)
if items.size
== 1:
    return 0
counts =
np.zeros((items.shape[0], 1))
sums = 0
for x in range(items.shape[0]):
    counts[x] = np.sum(S == items[x]) /
(S.size * 1.0)
for count in counts:
    sums += -1 * count *
math.log(count, 2)
return sums

```

```

def gain_ratio(data, col):
    items, dictionary = subtables(data, col,
delete=False)
total_size = data.shape[0]
entropies =
np.zeros((items.shape[0], 1))
intrinsic =
np.zeros((items.shape[0], 1))
for
x in range(items.shape[0]):
    ratio = dictionary[items[x]].shape[0] / (total_size
* 1.0)
    entropies[x] = ratio *
entropy(dictionary[items[x]][:, -1])
    intrinsic[x] =
ratio * math.log(ratio, 2)
total_entropy =
entropy(data[:, -1])
iv = -1 *
sum(intrinsic)
for x in
range(entropies.shape[0]
):
    total_entropy -=
entropies[x]
return total_entropy / iv

```

```

def create_node(data, metadata):
    if (np.unique(data[:, -
1])).shape[0] == 1:
        node =
Node("")
        node.answer =
np.unique(data[:, -1])[0]
        return
        node
    gains = np.zeros((data.shape[1]
- 1, 1))
    for col in
range(data.shape[1] - 1):
        gains[col] =

```

```

gain_ratio(data, col)split =
np.argmax(gains)
node = Node(metadata[split])
metadata = np.delete(metadata, split, 0)
items, dictionary = subtables(data, split,
delete=True)for x in range(items.shape[0]):
    child = create_node(dictionary[items[x]],
    metadata)node.children.append((items[x],
    child))
return
node def
empty(siz
ze):
    s = ""
    for x in
        range(siz
e):s += "
    "
    return s

def
print_tree(node,
level):if
node.answer !=
"":
    print(empty(level),
    node.answer)return
print(empty(level),
node.attribute)for value, n
in node.children:
    print(empty(level +
    1), value)print_tree(n,
    level + 2)

metadata, traindata =
read_data("book1.csv")data =
np.array(traindata)
node = create_node(data,
metadata)print_tree(node,
0)

```

Dataset: (tennisdata.csv)

Outlook, Temperature, Humidity, Windy, Play
Tennis Sunny, Hot, High, False, No
Sunny, Hot, High, True, No

Overcast,Hot,High,False,Yes
Rainy,Mild,High,False,Yes
Rainy,Cool,Normal,False,Yes
Rainy,Cool,Normal,True,No
Overcast,Cool,Normal,True,Yes
Sunny,Mild,High,False,No
Sunny,Cool,Normal,False,Yes
Rainy,Mild,Normal,False,Yes
Sunny,Mild,Normal,True,Yes
Overcast,Mild,High,True,Yes
Overcast,Hot,Normal,False,Yes
Rainy,Mild,High,True,No

RESULT:

Thus the Implementation for ID3Algorithm using Python Program has been executed successfully and output also verified

Exp.No:

Date:

BACKPROPAGATION ALGORITHM

LEARNING OBJECTIVES:

- Make use of Data sets in implementing the machine learning algorithms.
- Implement ML concepts and algorithms in Python

AIM:

- To Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

THEORY:

- Artificial neural networks (ANNs) provide a general, practical method for learning real-valued, discrete-valued, and vector-valued functions from examples.
- Algorithms such as BACKPROPAGATION gradient descent to tune network parameters to best fit a training set of input-output pairs.
- ANN learning is robust to errors in the training data and has been successfully applied to problems such as interpreting visual scenes, speech recognition, and learning robot control strategies.

Backpropagation Algorithm:

1. Create a feed-forward network with n_i inputs, n_h hidden units, and n_o output units.
2. Initialize each w_i to some small random value (e.g., between -0.05 and 0.05).
3. Until the termination condition is met, do

For each training example $\langle (x_1, \dots, x_n), t \rangle$, do

// Propagate the input forward through the network:

a. Input the instance $(x_1, \dots, x_n)$ to the n/w & compute the n/w outputs o_k for every unit

// Propagate the errors backward through the network:

b. For each output unit k , calculate its error term δ_k ; $\delta_k = o_k(1-o_k)(t_k-o_k)$

c. For each hidden unit h , calculate its error term δ_h ; $\delta_h = o_h(1-o_h) \sum_k w_{h,k} \delta_k$

For each network weight $w_{i,j}$ do; $w_{i,j} = w_{i,j} + \delta_i w_{h,k} x_{i,j}$ where $w_{i,j} = \sum_j x_{i,j} \delta_j$

PROGRAM:

```
import numpy as np
X = np.array([[2, 9], [1, 5], [3, 6]], dtype=float)
Y = np.array([92, 86, 89],
dtype=float)X = X / np.amax(X,
axis=0)
Y = Y / 100

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

epoch = 7000
lr = 0.1
inputlayer_neurons = 2
hiddenlayer_neurons = 3
output_neurons = 1

wh = np.random.uniform(size=(inputlayer_neurons,
hiddenlayer_neurons))bh = np.random.uniform(size=(1,
hiddenlayer_neurons))
wout = np.random.uniform(size=(hiddenlayer_neurons,
output_neurons))bout = np.random.uniform(size=(1,
output_neurons))

for i in
    range(epoch):
        hinp1 =
        np.dot(X, wh)
        hinp = hinp1 +
        bh
        hlayer_act = sigmoid(hinp)
        outinp1 =
        np.dot(hlayer_act, wout)
        outinp = outinp1 + bout
        output = sigmoid(outinp)

EO = Y - output
outgrad = output * (1 - output) # Corrected sigmoid
derivatived_output = EO * outgrad
EH = d_output.dot(wout.T)
hiddengrad = hlayer_act * (1 - hlayer_act) # Corrected sigmoid derivative
d_hiddenlayer = EH * hiddengrad
```

```

wout +=
hlayer_act.T.dot(d_output) * lr
wh += X.T.dot(d_hiddenlayer)
* lr

if i % 1000 == 0:
    print(f'Epoch {i}: Error = {np.mean(np.abs(EO))}')
print("Input: \n", X)
print("Actual Output:
\n", Y)
print("Predicted Output: \n", output)

```

OUTPUT:

```

Input:
[[0.66666667 1.          ]
 [0.33333333 0.55555556]
 [1.          0.66666667]]
Actual Output:
[[0.92]
 [0.86]
 [0.89]]
Predicted Output:
[[0.75495298]
 [0.74126699]
 [0.75392083]]

```

RESULT:

Thus the Implementation for Backpropagation using Python Program has been executed successfully and output also verified.

Exp.No:

Date:

NAÏVE BAYESIAN CLASSIFIER

LEARNING OBJECTIVES:

- Make use of Data sets in implementing the machine learning algorithms.
- Implement ML concepts and algorithms in Python

AIM:

To write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.

THEORY:

Naive Bayes Algorithm:

Naive Bayes algorithm is a classification technique based on Bayes 'Theorem with an assumption of independence among predictors. In simple terms, a Naive Bayes classifier assumes that the presence of a particular feature in a class is unrelated to the presence of any other feature. For example, a fruit may be considered to be an apple if it is red, round, and about 3 inches in diameter. Even if these features depend on each other or upon the existence of the other features, all of these properties independently contribute to the probability that this fruit is an apple and that is why it is known as 'Naive'.

Naive Bayes model is easy to build and particularly useful for very large data sets. Along with simplicity, Naive Bayes is known to outperform even highly

The diagram shows the formula for the posterior probability of class c given predictor x :

$$P(c | x) = \frac{P(x | c)P(c)}{P(x)}$$

Labels with arrows pointing to the formula components:

- Likelihood** points to $P(x | c)$.
- Class Prior Probability** points to $P(c)$.
- Posterior Probability** points to $P(c | x)$.
- Predictor Prior Probability** points to $P(x)$.

$$P(c | X) = P(x_1 | c) \times P(x_2 | c) \times \dots \times P(x_n | c) \times P(c)$$
sophisticated classification methods.

Bayes theorem provides a way of calculating posterior probability $P(c|x)$ from $P(c)$, $P(x)$ and $P(x|c)$. Look at the equation below where $P(c|x)$ is the posterior probability of class (c , target) given predictor (x , attributes). $P(c)$ is the prior

probability of class.

$P(x|c)$ is the likelihood which is the probability of predictor given class. $P(x)$ is the prior probability of predictor.

The naive Bayes classifier applies to learning tasks where each instance x is described by a conjunction of attribute values and where the target function $f(x)$ can take on any value from some finite set V . A set of training examples of the target function is provided, and a new instance is presented, described by the tuple of attribute values $(a_1, a_2, \dots, a_n)$. The learner is asked to predict the target value, or classification, for this new instance.

The Bayesian approach to classifying the new instance is to assign the most probable target value v_{MAP} , given the attribute values $(a_1, a_2, \dots, a_n)$ that describe the instance.

$$v_{MAP} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j | a_1, a_2, \dots, a_n)$$

We can use Bayes theorem to rewrite this expression as

$$\begin{aligned} v_{MAP} &= \underset{v_j \in V}{\operatorname{argmax}} \frac{P(a_1, a_2, \dots, a_n | v_j) P(v_j)}{P(a_1, a_2, \dots, a_n)} \\ &= \underset{v_j \in V}{\operatorname{argmax}} P(a_1, a_2, \dots, a_n | v_j) P(v_j) \end{aligned}$$

Now we could attempt to estimate the two terms in Equation (19) based on the training data. It is easy to estimate each of the $P(v_j)$ simply by counting the frequency with which each target value v_j occurs in the training data.

The naive Bayes classifier is based on the simplifying assumption that the attribute values are conditionally independent given the target value. In other words, the assumption is that given the target value of the instance, the probability of observing the conjunction $a_1, a_2, \dots, a_n$, is just the product of the probabilities for the individual attributes: $P(a_1, a_2, \dots, a_n | v_j) = \prod_i P(a_i | v_j)$. Substituting this, we have the approach used by the naive Bayes classifier.

$$v_{NB} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j) \prod_i P(a_i | v_j)$$

where v_{NB} denotes the target value output by the naive Bayes classifier.

When dealing with continuous data, a typical assumption is that the continuous values associated with each class are distributed according to a Gaussian distribution. For

example, suppose the training data contains a continuous attribute, x . We first segment the data by the class, and then compute the mean and variance of x in each class.

Let μ be the mean of the values in x associated with class C_k , and let σ_k^2 be the variance of the values in x associated with class C_k . Suppose we have collected some observation

$$p(x = v | C_k) = \frac{1}{\sqrt{2\pi\sigma_k^2}} e^{-\frac{(v-\mu_k)^2}{2\sigma_k^2}}$$

value v . Then, the probability distribution of v given a class C_k , $p(x=v|C_k)$ can be computed by plugging v into the equation for a Normal distribution parameterized by μ and σ_k^2 . That is above method is adopted in our implementation of the Program. Pima Indian diabetes dataset

This dataset is originally from the National Institute of Diabetes and Digestive and Kidney Diseases. The objective of the dataset is to diagnostically predict whether or not a patient has diabetes, based on certain diagnostic measurements included in the dataset.

Dataset:

Outlook, Temperature, Humidity, Windy, P
 ay Tennis Sunny, Hot, High, False, No
 Sunny, Hot, High, True, No
 Overcast, Hot, High, False, Yes
 Rainy, Mild, High, False, Yes
 Rainy, Cool, Normal, False, Yes
 Rainy, Cool, Normal, True, No
 Overcast, Cool, Normal, True, Yes
 Sunny, Mild, High, False, No
 Sunny, Cool, Normal, False, Yes
 Rainy, Mild, Normal, False, Yes
 Sunny, Mild, Normal, True, Yes
 Overcast, Mild, High, True, Yes
 Overcast, Hot, Normal, False, Yes
 Rainy, Mild, High, True, No

PROGRAM:

```
In [11]: import pandas as pd
from sklearn import tree
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
```

```
In [12]: data = pd.read_csv('tennisdata.csv')
print("The first 5 values of data is :\n", data.head())
```

The first 5 values of data is :

	Outlook	Temperature	Humidity	Windy	PlayTennis
0	Sunny	Hot	High	False	No
1	Sunny	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Rainy	Mild	High	False	Yes
4	Rainy	Cool	Normal	False	Yes

```
In [13]: X = data.iloc[:, :-1]
print("\nThe First 5 values of train data is\n", X.head())
```

The First 5 values of train data is

	Outlook	Temperature	Humidity	Windy
0	Sunny	Hot	High	False
1	Sunny	Hot	High	True
2	Overcast	Hot	High	False
3	Rainy	Mild	High	False
4	Rainy	Cool	Normal	False

```
In [15]: y = data.iloc[:, -1]
print("\nThe first 5 values of Train output is\n", y.head())
```

The first 5 values of Train output is

0	No
1	No
2	Yes
3	Yes
4	Yes

Name: PlayTennis , dtype: object

```
In [16]: le_outlook = LabelEncoder()
X.Outlook=le_outlook.fit_transform(X.Outlook)

le_Temperature=LabelEncoder()
X.Temperature = le_Temperature.fit_transform(X. Temperature)

le_Humidity=LabelEncoder()
X.Humidity=le_Humidity.fit_transform(X.Humidity)

le_Windy = LabelEncoder()
X.Windy=le_Windy.fit_transform(X.Windy)

print("\nNow the Train data is : \n",X.head())
```

```
Now the Train data is :
   Outlook  Temperature  Humidity  Windy
0        2             1         0      0
1        2             1         0      1
2        0             1         0      0
3        1             2         0      0
4        1             0         1      0
```

```
In [17]: le_PlayTennis=LabelEncoder()
y=le_PlayTennis.fit_transform(y)
print("\nNow the Train output is\n",y)
```

```
Now the Train output is
[0 0 1 1 1 0 1 0 1 1 1 1 0]
```

```
In [19]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.20)

classifier = GaussianNB()
classifier.fit(X_train,y_train)

from sklearn.metrics import accuracy_score
print("Accuracy is: ", accuracy_score (classifier.predict(X_test),y_test))

Accuracy is:  0.6666666666666666
```

RESULT:

Thus the Implementation for Naïve Bayesian Classifier using Python Program has been executed successfully and output also verified.

Exp.No:

Date:

DOCUMENT CLASSIFICATION USING NAÏVE BAYESIAN CLASSIFIER

LEARNING OBJECTIVES:

- Make use of Data sets in implementing the machine learning algorithms.
- Implement ML concepts and algorithms in Python.

AIM:

To Assuming a set of documents that need to be classified, use the naïve Bayesian Classifier model to perform this task. Built-in Java classes/API can be used to write the program. Calculate the accuracy, precision, and recall for your data set.

THEORY:

For the theory of the naive Bayesian classifier refer Experiment No. 5. Theory of performance analysis is elaborated here.

ANALYSIS OF DOCUMENT CLASSIFICATION

		Predicted	
		Negative	Positive
Actual	Negative	True Negative	False Positive
	Positive	False Negative	True Positive

- For classification tasks, the terms true positives, true negatives, false positives, and false negatives compare the results of the classifier under test with trusted external judgments. The terms positive and negative refer to the classifier's prediction (sometimes known as the expectation), and the terms true and false refer to whether that prediction corresponds to the external judgment (sometimes known as the observation).
- Precision - Precision is the ratio of correctly predicted positive documents to the total predicted positive documents. High precision relates to the low false positive rate.
- Precision = $(\sum \text{True positive}) / (\sum \text{True positive} + \sum \text{False positive})$
- Recall (Sensitivity) - Recall is the ratio of correctly predicted positive documents to the all observations in actual class.
- Recall = $(\sum \text{True positive}) / (\sum \text{True positive} + \sum \text{False negative})$

- Accuracy - Accuracy is the most intuitive performance measure and it is simply a ratio of correctly predicted observation to the total observations. One may think that, if we have high accuracy then our model is best. Yes, accuracy is a great measure but only when you have symmetric datasets where values of false positive and false negatives are almost same. Therefore, you have to look at other parameters to evaluate the performance of your model. For our model, we have got 0.803 which means our model is approx. 80% accurate.
- $\text{Accuracy} = (\Sigma \text{ True positive} + \Sigma \text{ True negative}) / \Sigma \text{ Total population}$

DATASET (document.csv)

I love this sandwich,
 pos This is an
 amazing place,pos
 I feel very good about these
 beers,posThis is my best
 work,pos
 What an awesome view,pos
 I do not like this
 restaurant,negI am
 tired of this stuff,neg
 I can't deal with
 this,neg He is my
 sworn enemy,neg
 My boss is
 horrible,neg
 This is an awesome place,pos
 I do not like the taste of this
 juice,negI love to dance,pos
 I am sick and tired of this
 place,negWhat a great
 holiday,pos
 That is a bad locality to stay,neg
 We will have good fun
 tomorrow,pos I went to my
 enemy's house today,neg

PROGRAM:

```
In [1]: import pandas as pd
msg = pd.read_csv('document.csv', names=['message', 'label'])
print("Total Instances of Dataset: ", msg.shape[0])
msg['labelnum'] = msg.label.map({'pos': 1, 'neg': 0})
```

Total Instances of Dataset: 18

```
In [3]: x = msg.message
y = msg.labelnum
from sklearn.model_selection import train_test_split
Xtrain, Xtest, ytrain, ytest = train_test_split(x, y)
from sklearn.feature_extraction.text import CountVectorizer

count_v = CountVectorizer()
Xtrain_dm = count_v.fit_transform(Xtrain)
Xtest_dm = count_v.transform(Xtest)
```

```
In [4]: df = pd.DataFrame(Xtrain_dm.toarray(), columns=count_v.get_feature_names_out())
print(df[0:5])
```

	about	am	an	and	awesome	bad	beers	best	boss	dance	...	these
0	1	0	0	0	0	0	1	0	0	0	...	1
1	0	0	0	0	0	1	0	0	0	0	...	0
2	0	0	0	0	0	0	0	0	0	0	...	0
3	0	0	0	0	0	0	0	1	0	0	...	0
4	0	0	0	0	0	0	0	0	0	1	...	0

	this	tired	to	today	very	view	went	what	work
0	0	0	0	0	1	0	0	0	0
1	0	0	1	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0
3	1	0	0	0	0	0	0	0	1
4	0	0	1	0	0	0	0	0	0

[5 rows x 45 columns]

```
In [6]: from sklearn.naive_bayes import MultinomialNB
        clf = MultinomialNB()
        clf.fit(Xtrain_dm, ytrain)
        pred = clf.predict(Xtest_dm)
```

```
In [7]: for doc, p in zip (Xtrain, pred):
        p = 'pos' if p == 1 else 'neg'
        print("%s ->%s" % (doc, p))
```

```
I feel very good about these beers ->pos
That is a bad locality to stay ->neg
I do not like the taste of this juice ->pos
This is my best work ->pos
I love to dance ->neg
```

```
In [9]: from sklearn.metrics import accuracy_score, confusion_matrix, precision_score
        print('Accuracy Metrics: \n')
        print('Accuracy: ', accuracy_score(ytest, pred))
        print('recall: ', recall_score(ytest, pred))
        print('Precision: ', precision_score(ytest, pred))
        print('Confusion Matrix: \n', confusion_matrix(ytest, pred))
```

Accuracy Metrics:

```
Accuracy:  0.8
recall:  0.75
Precision:  1.0
Confusion Matrix:
[[1 0]
 [1 3]]
```

RESULT:

Thus the Implementation for Document Classification Using Naïve Bayesian Classifier Python Program has been executed successfully and output also verified.

Exp.No:

Date:

BAYESIAN NETWORK

LEARNING OBJECTIVES:

- Make use of Data sets in implementing the machine learning algorithms.
- Implement ML concepts and algorithms in Python.

AIM:

To write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using standard Heart Disease Data Set. You can use Java/Python ML library classes/API

THEORY:

- Bayesian networks are very convenient for representing similar probabilistic relationships between multiple events.
- Bayesian networks as graphs - People usually represent Bayesian networks as directed graphs in which each node is a hypothesis or a random process. In other words, something that takes at least 2 possible values you can assign probabilities to. For example, there can be a node that represents the state of the dog (barking or not barking at the window), the weather (raining or not raining), etc.
- The arrows between nodes represent the conditional probabilities between them how information about the state of one node changes the probability distribution of another node it's connected to.

Dataset (heart_disease.csv)

```
Age,Gender,Family,diet,Lifestyle,cholesterol,heartdisease
0,0,1,1,3,0,1
0,1,1,1,3,0,1
1,0,0,0,2,1,1
4,0,1,1,3,2,0
3,1,1,0,0,2,0
2,0,1,1,1,0,1
4,0,1,0,2,0,1
0,0,1,1,3,0,1
3,1,1,0,0,2,0
1,1,0,0,0,2,1
4,1,0,1,2,0,1
4,0,1,1,3,2,0
2,1,0,0,0,0,0
```

2,0,1,1,1,0,1
3,1,1,0,0,1,0
0,0,1,0,0,2,1

1,1,0,1,2,1,1
3,1,1,1,0,1,0
4,0,1,1,3,2,0

Program:

```
import pandas as pd
data=pd.read_csv("heart_disease.csv")
heart_disease=pd.DataFrame(data)
print (heart_disease)
from pgmpy.models import BayesianNetwork
model=BayesianModel([
('age', 'Lifestyle'),
('Gender', 'Lifestyle'),
('Family', 'heartdisease'),
('diet', 'cholesterol'),
('Lifestyle', 'diet'),
('cholesterol', 'heartdisease'),('diet',
'cholesterol')])
from pgmpy.estimators import MaximumLikelihoodEstimator
model.fit(heart_disease,estimator=MaximumLikelihoodEstimator)
from pgmpy.inference import VariableElimination
HeartDisease_infer = VariableElimination(model)
print('For age Enter { SuperSenior Citizen: 0, Senior Citizen: 1, MiddleAged: 2, Youth: 3,Teen: 4 }')
print('For Gender Enter { Male:0, Female: 1 }')
print('For Family History Enter { yes:1, No: 0 }')
print('For diet Enter { High:0, Medium: 1 }')
print('For lifeStyle Enter { Athlete:0, Active:1, Moderate: 2, Sedentary: 3 }')
print('For cholesterol Enter { High:0, Borderline:1, Normal: 2 }')
q= HeartDisease_infer.query(variables=['heart disease'],
```

```
evidence={'age': int(input('Enter age :')),
'Gender': int(input('Enter Gender
:')), 'Family': int(input('Enter Family
history :')), 'diet':int(input('Enter diet
:')), 'Lifestyle':int(input('Enter
Lifestyle :')), 'cholesterol': int
(input('Enter cholesterol :'))
})
print(q)
```

OUTPUT:

	age	Gender	Family	diet	Lifestyle	cholesterol	heartdisease
0	0	0	1	1	3	0	1
1	0	1	1	1	3	0	1
2	1	0	0	0	2	1	1
3	4	0	1	1	3	2	0
4	3	1	1	0	0	2	0
5	2	0	1	1	1	0	1
6	4	0	1	0	2	0	1
7	0	0	1	1	3	0	1
8	3	1	1	0	0	2	0
9	1	1	0	0	0	2	1
10	4	1	0	1	2	0	1
11	4	0	1	1	3	2	0
12	2	1	0	0	0	0	0
13	2	0	1	1	1	0	1
14	3	1	1	0	0	1	0
15	0	0	1	0	0	2	1
16	1	1	0	1	2	1	1
17	3	1	1	1	0	1	0
18	4	0	1	1	3	2	0

For age Enter { SuperSeniorCitizen:0, SeniorCitizen:1, MiddleAged:2, Youth:3, Teen:4 }
For Gender Enter { Male:0, Female:1 }
For Family History Enter { yes:1, No:0 }
For diet Enter { High:0, Medium:1 }
For lifeStyle Enter { Athlete:0, Active:1, Moderate:2, Sedentary:3 }
For cholesterol Enter { High:0, BorderLine:1, Normal:2 }
Enter age :1
Enter Gender :1
Enter Family history :0
Enter diet :1
Enter Lifestyle :0
Enter cholesterol :1

heartdisease	phi(heartdisease)
heartdisease(0)	0.0000
heartdisease(1)	1.0000

RESULT:

Thus the Implementation for Bayesian Network using Python Program has been executed successfully and output also verified.

Exp.No:

Date:

CLUSTERING BASED ON EXPECTATION MAXIMUM AND K-MEANS ALGORITHM

LEARNING OBJECTIVES:

- Make use of Data sets in implementing the machine learning algorithms.
- Implement ML concepts and algorithms in Python

AIM:

To Apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same data set for Clustering using k-Means algorithm. Compare the results of these two algorithms and comment on the quality of clustering. You can add Java/Python ML library classes/API in the program.

THEORY: Expectation Maximization algorithm

- The basic approach and logic of this clustering method is as follows.
- Suppose we measure a single continuous variable in a large sample of observations. Further, suppose that the sample consists of two clusters of observations with different means (and perhaps different standard deviations); within each sample, the distribution of values for the continuous variable follows the normal distribution.
- The goal of EM clustering is to estimate the means and standard deviations for each clusters as to maximize the likelihood of the observed data (distribution).
- Put another way, the EM algorithm attempts to approximate the observed distributions of values based on mixtures of different distributions in different clusters. The results of EM clustering are different from those computed by k-means clustering.
- The latter will assign observations to clusters to maximize the distances between clusters. The EM algorithm does not compute actual assignments of observations to clusters, but classification probabilities.
- In other words, each observation belongs to each cluster with a certain probability. Of course, as a final result we can usually review an actual assignment of observations to clusters, based on the (largest) classification probability.

K means Clustering

- The algorithm will categorize the items into k groups of similarity. To calculate that similarity, we will use the Euclidean distance as measurement.
- The algorithm works as follows:
 1. First we initialize k points, called means, randomly.
 2. We categorize each item to its closest mean and we update the mean's coordinates, which are the averages of the items categorized in that mean so far.
 3. We repeat the process for a given number of iterations and at the end, we have our clusters.
- The points mentioned above are called means, because they hold the mean values of the items categorized in it. To initialize these means, we have a lot of options. An intuitive method is to initialize the means at random items in the data set. Another method is to initialize the means at random values between the boundaries of the data set (if for a feature the items have values in [0,3], we will initialize the means with values for x at [0,3]).
- Pseudo code:
 1. Initialize k means with random values
 2. For a given number of iterations: Iterate through items:
 - Find the mean closest to the item
 - Assign item to update mean

PROGRAM:

```
from sklearn.cluster import
KMeansfrom sklearn import
preprocessing
from sklearn.mixture import
GaussianMixturefrom sklearn.datasets
import load_iris
import
sklearn.metrics as
smimport pandas as
pd
import numpy as np
import
matplotlib.pyplot as
pltdataset=load_iris()
# print(dataset)
X=pd.DataFrame(datas
et.data)
X.columns=['Sepal_Length', 'Sepal_Width', 'Petal_Length',
'Petal_Width']y=pd.DataFrame(dataset.target)
```

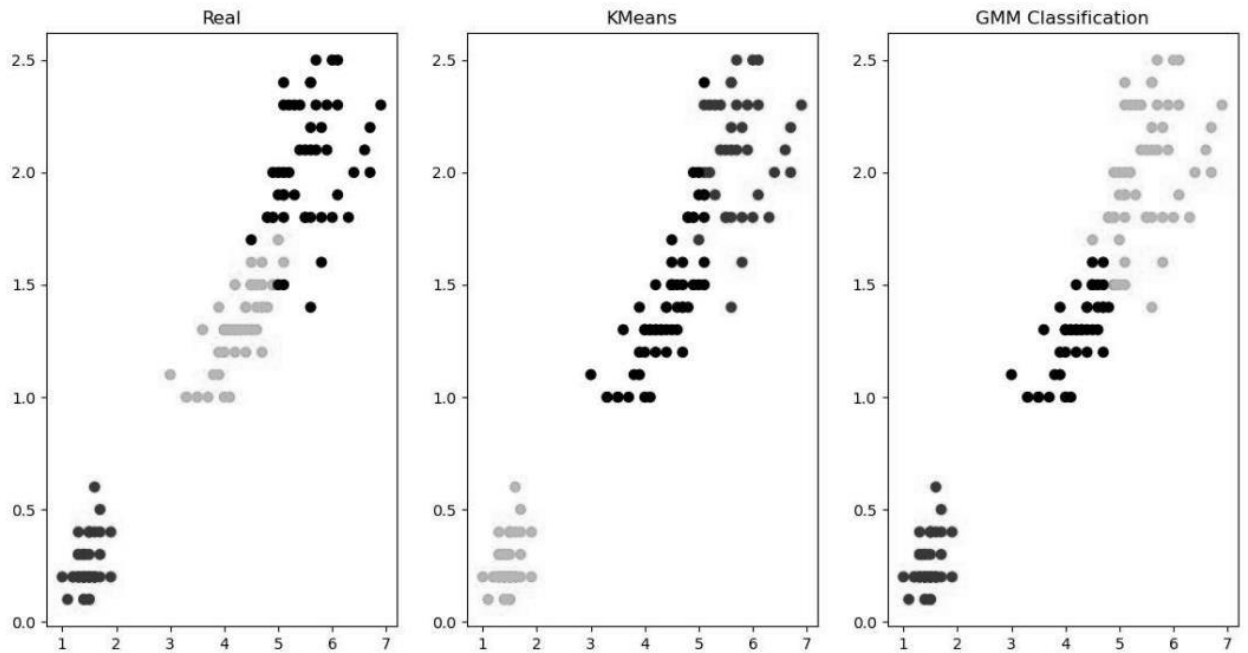
```

y.columns=['
Targets']#
print(X)
plt.figure(figsize=(14,7))
colormap=np.array([ 'red', 'lime',
'black'])# REAL PLOT
plt.subplot(1,3,1)
plt.scatter(X.Petal_Length, X. Petal_Width, c=colormap[y.
Targets], s=40)plt.title('Real')
# K-PLOT
plt.subplot(1,3,2)
model=KMeans
(n_clusters=3)
model.fit(X)
predY=np.choose (model.labels_, [0,1,2]).astype (np.int64)
plt.scatter(X.Petal_Length, X. Petal_Width,
c=colormap[predY], s=40)plt.title('KMeans')
#GMM PLOT
scaler=preprocessing.
StandardScaler()scaler.fit(X)
xsa=scaler.transform(X)
xs=pd.DataFrame(xsa,
columns=X.columns)
gmm=GaussianMixture
(n_components=3) gmm.fit(xs)
y_cluster_gmm=gmm.predict(xs)
plt.subplot(1,3,3)
plt.scatter (X. Petal_Length, X. Petal_Width,
c=colormap[y_cluster_gmm], s=40)plt.title('GMM Classification')

```

OUTPUT:

```
Out[2]: Text(0.5, 1.0, 'GMM Classification')
```



RESULT:

Thus the Implementation for Clustering Based on Expectation Maximum and K-Means Algorithm using Python Program has been executed successfully and output also verified.

Exp.No:

Date:

KNN ALGORITHM

LEARNING OBJECTIVES:

- Make use of Data sets in implementing the machine learning algorithms.
- Implement ML concepts and algorithms in Python.

AIM:

To write a program to implement k-Nearest Neighbor algorithm to classify the iris data set. Print both correct and wrong predictions. Java/Python ML library classes can be used for this problem.

THEORY:

- K-Nearest Neighbors is one of the most basic yet essential classification algorithms in Machine Learning. It belongs to the supervised learning domain and finds intense application in pattern recognition, data mining and intrusion detection.
- It is widely disposable in real-life scenarios since it is non-parametric, meaning, it does not make any underlying assumptions about the distribution of data.
- Algorithm

Input: Let m be the number of training data samples. Let p be an unknown point. Method:

1. Store the training samples in an array of data points $arr[]$. This means each element of this array represents a tuple (x, y) .
 2. for $i=0$ to m
Calculate Euclidean distance $d(arr[i], p)$.
 3. Make set S of K smallest distances obtained. Each of these distances correspond to an already classified data point.
- Return the majority label among S .

PROGRAM:

```
from sklearn.datasets import load_iris
from sklearn.neighbors import
KNeighborsClassifierfrom
sklearn.model_selection import
train_test_splitimport numpy as np
dataset=lo
ad_iris()
#print(data
set)
```

```

X_train,X_test,y_train, y_test=train_test_split(dataset ["data"], dataset ["target"],
random_state=0)kn=KNeighborsClassifier (n_neighbors=1)
kn.fit(X_train,y_train)
KNeighborsClassifier(algorithm= 'auto', leaf_size=30,
metric='minkowski',metric_params=None, n_jobs=None,
n_neighbors =1, p=2,
weights=' uniform')
for i in
    range(len(X_test)):
        x=X_test[i]
        x_new=np.array([x])
        prediction=kn.predict(
            x_new)
        print("TARGET=",y_test[i], dataset["target_names"][y_test[i]],"PREDICTED=",
prediction,dataset["target_names"] [prediction])
print (kn.score (X_test,y_test))

```

OUTPUT:

```

TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']

```

TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 1 versicolor PREDICTED= [2] ['virginica']
0.9736842105263158

RESULT:

Thus the Implementation for KNN Algorithm using Python Program has been executed successfully and output also verified.

Exp.No:

Date:

LOCALLY WEIGHTED REGRESSIONALGORITHM

LEARNING OBJECTIVES:

- Make use of Data sets in implementing the machine learning algorithms.
- Implement ML concepts and algorithms in Python.

AIM:

To implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

THEORY:

- Given a dataset X, y , we attempt to find a linear model $h(x)$ that minimizes residual sum of squared errors. The solution is given by Normal equations.
- Linear model can only fit a straight line, however, it can be empowered by polynomial features to get more powerful models. Still, we have to decide and fix the number and types of features ahead.
- Alternate approach is given by locally weighted regression.
- Given a dataset X, y , we attempt to find a model $h(x)$ that minimizes residual sum of weighted squared errors.
- The weights are given by a kernel function which can be chosen arbitrarily and in my case I choose a Gaussian kernel.
- The solution is very similar to Normal equations, we only need to insert diagonal weight matrix W .

ALGORITHM :

```
def local_regression(x0, X, Y, tau):#
    add bias term x0 = np.r_[1, x0]
    X = np.c_[np.ones(len(X)), X]

    # fit model: normal equations with kernel xw = X.T
    * radial_kernel(x0, X, tau)
    beta = np.linalg.pinv(xw @ X) @ xw @ Y

    # predict value return
    x0 @ beta def
    radial_kernel(x0, X,
    tau):
    return np.exp(np.sum((X - x0) ** 2, axis=1) / (-2 * tau * tau))
```

PROGRAM :

```
from math
import ceil
import numpy as
np from scipy
import linalg

def lowess (x, y, f,
            iterations):n =
            len(x)
            r = int(ceil(f*n))
            h = [np.sort(np.abs(x - x[i]))[r] for i in range(n)]
            w = np.clip(np.abs ((x[:, None] -x[None, ]) / h),
            0.0, 1.0)w = (1 - w ** 3) ** 3
            yest
            =np.zeros(n
            ) delta =
            np.ones(n)
            for iteration in
            range(iterations):for i
            in range(n):
                weights=delta*w[:, i]
                b = np.array([np. sum (weights* y), np. sum (weights * y* x)])
                A = np.array([[np. sum (weights), np.sum (weights*x)], [np. sum (weights*x),
np.sum(weights *x*x)]]])
                beta=linalg.solve(A, b)
                yest[i] = beta[0] + beta[1] * x[i]

            residuals=y-yest
            s = np.median(np.abs(residuals))
            delta = np.clip (residuals /
            (6.0*s), -1, 1)delta=(1-delta **
            2) ** 2
            return yest

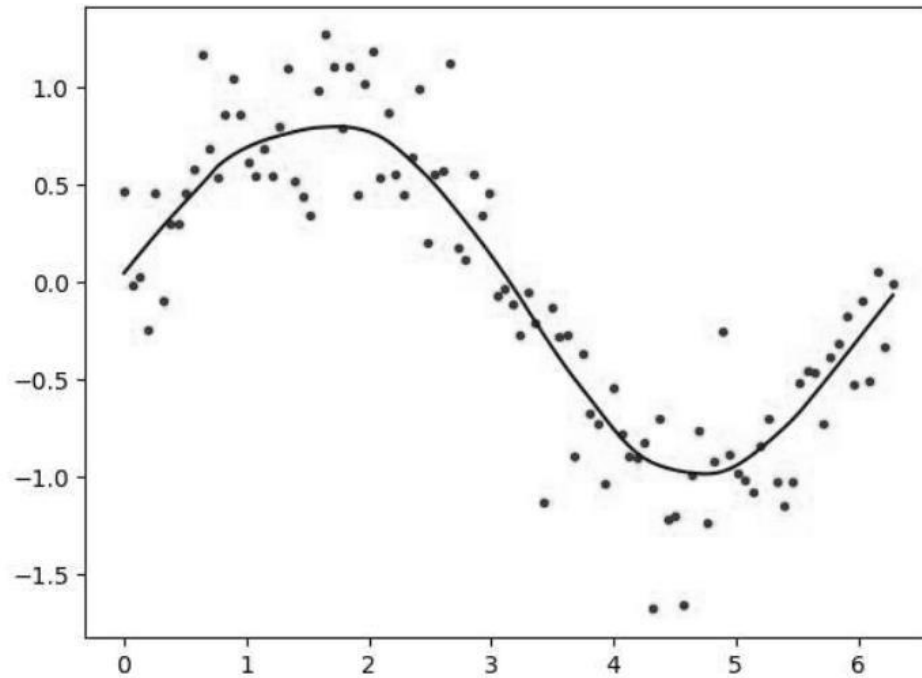
imp
ort
mat
hn =
100
x = np.linspace(0, 2*math.pi, n)
y =
np.sin(x)+0.3*np.random.rand
n(n)f =0.25
```

```
iterations=3  
yest= lowess (x, y, f, iterations)
```

```
import  
matplotlib.pyplot as  
pltplt.plot(x,y, "r.")  
plt.plot(x,yest, "b-")
```

OUTPUT:

```
Out[14]: [<matplotlib.lines.Line2D at 0x18b525c6280>]
```



RESULT:

Thus the Implementation for Locally Weighted Regression algorithm using Python Program has been executed successfully and output also verified.